

云原生架构分布式大模型推理的编排与调度实践

从 PD 分离到拓扑感知的全链路优化



Content 目录

01 破局：K8s 部署千亿大模型的三大挑战

02 重塑：基于 Kthena 的 LLM 声明式编排

03 深潜：多级拓扑感知与原子化 Gang 调度

04 实践：基于 vLLM-Ascend 的部署实践



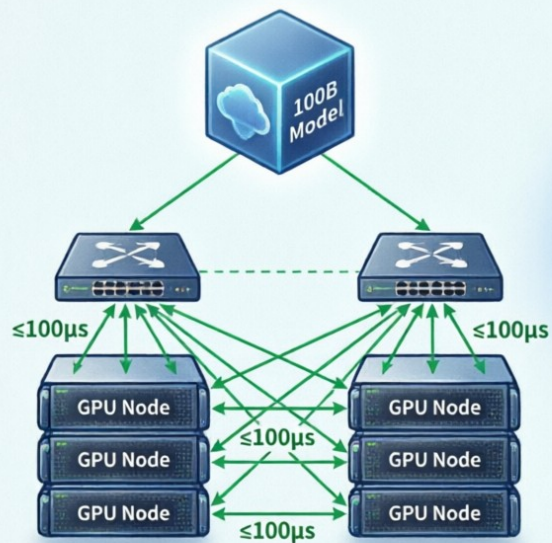
Part 01 破局：部署千亿大模型的三大挑战



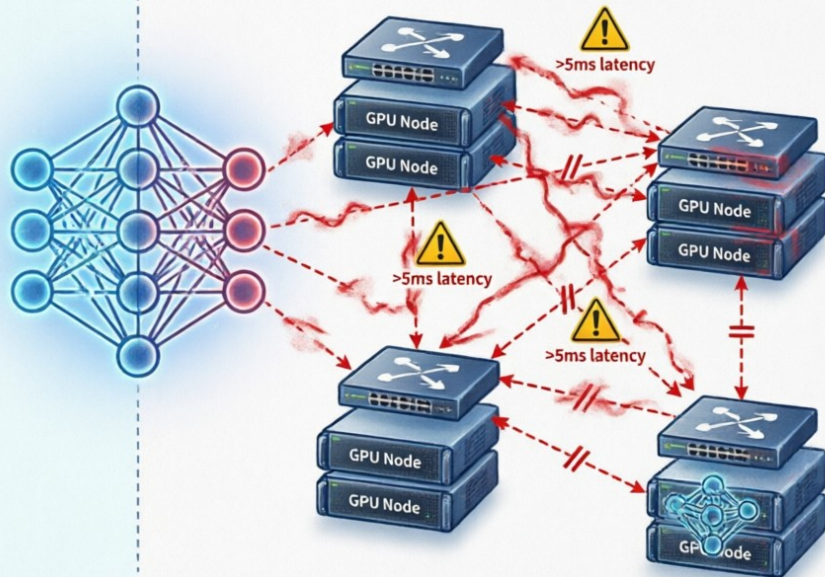
挑战一：多角色拓扑约束缺失导致通信延迟激增

Kubernetes集群中多角色拓扑约束缺失问题对比

理想拓扑感知部署



K8s原生随机调度问题



- 痛点：千亿参数模型需 TP/PP 混合并行，TP 对节点间（如 NVLink/RoCE）通信延迟极度敏感。
- K8s 原生困境：原生 Deployment/StatefulSet 无法表达多实例间的“亲和性与拓扑紧凑性”，导致 Pod 被随机打散，跨交换机通信引发延迟长尾。
- 后果：实例被随机打散，跨交换机通信引发严重的延迟长尾



挑战二：Prefill / Decode 分离架构下的编排弹性困难

LLM

Prefill/Decode分离架构的编排挑战

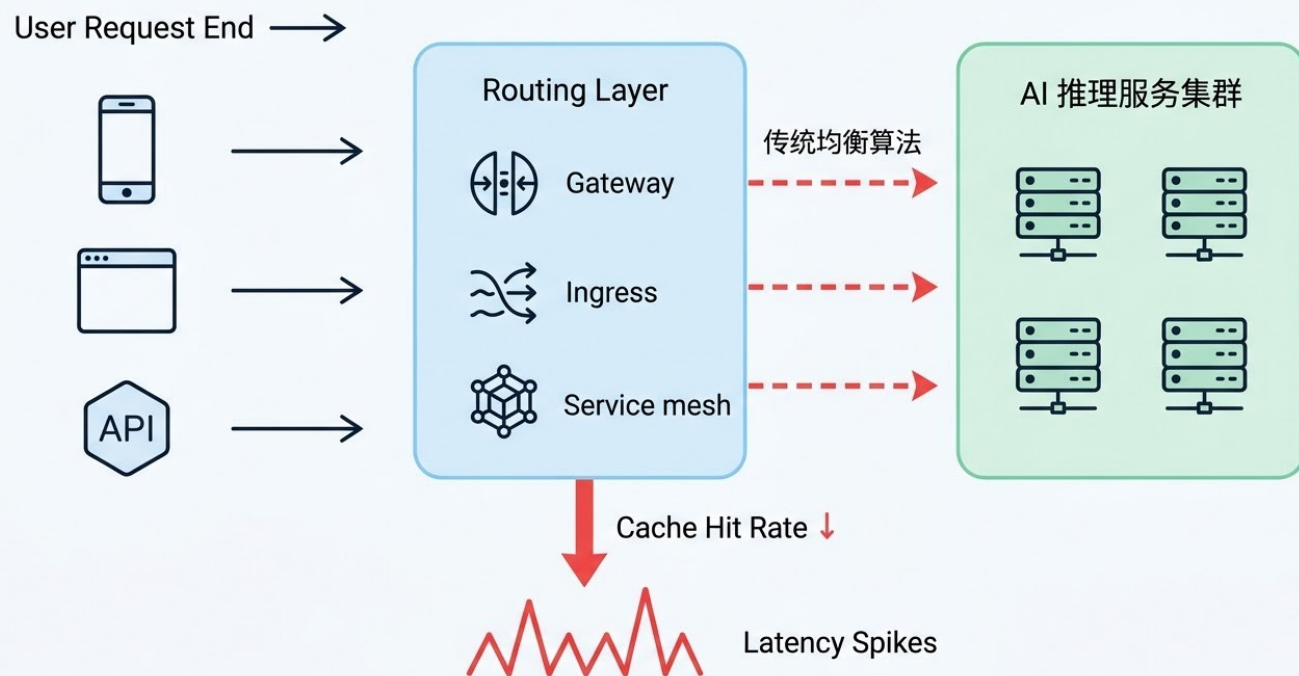


痛点：长上下文推理需采用分离架构，Prefill 与 Decode 模块需协同部署、独立弹性

K8s 原生困境：缺乏多实例 "原子化" 编排能力，无法实现联动伸缩

后果：部署复杂度高 · 运维管理困难 · 弹性伸缩受限

挑战三：请求调度时，AI 特有的均衡感知缺失



痛点：AI 推理负载具有独特的均衡需求，需综合历史负载、实时状态等多维度进行智能调度

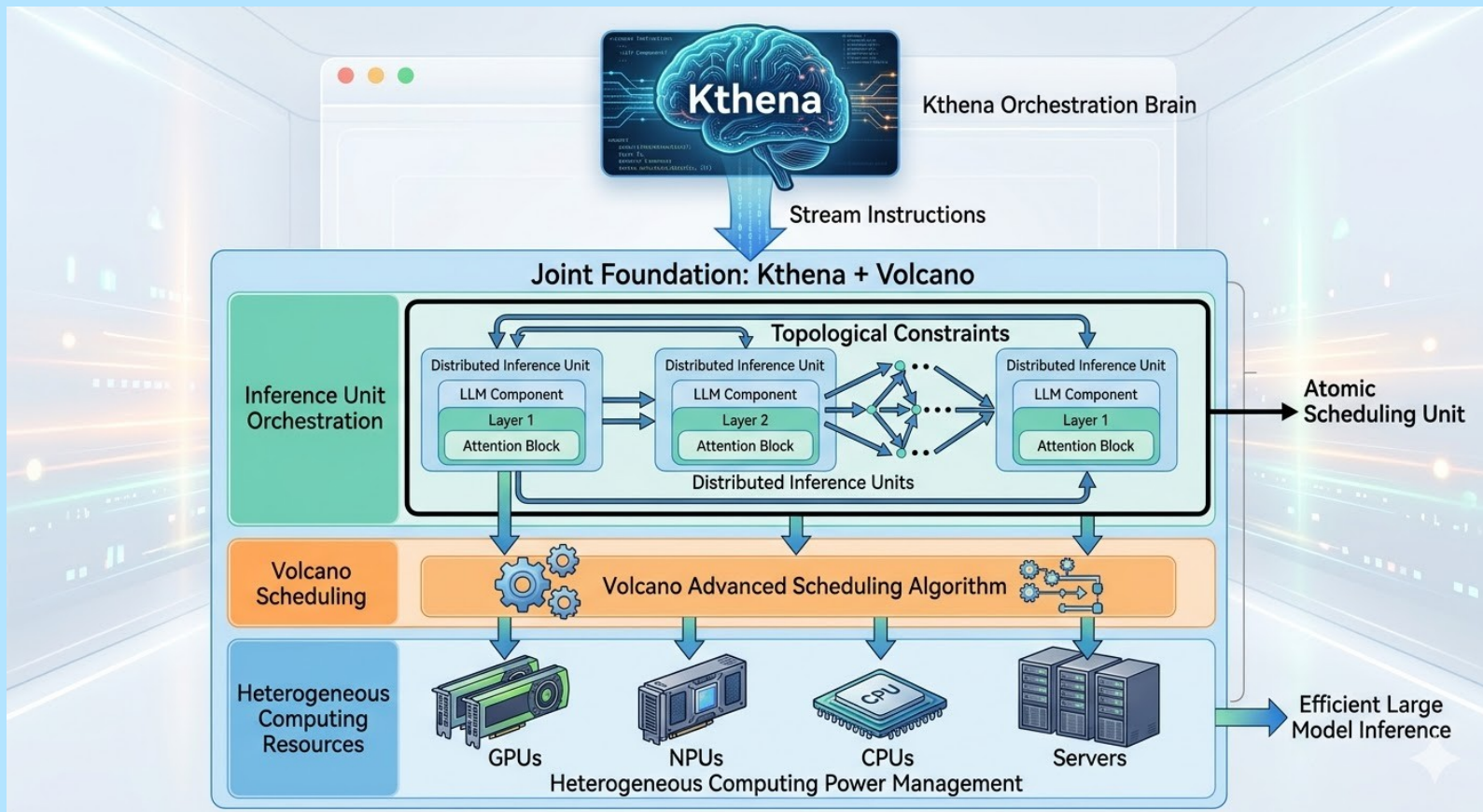
K8s 原生困境：原生路由层（Gateway/Ingress/Service）无法感知底层 AI 推理状态

后果：请求盲目分发 → 缓存命中率骤降 → 端到端延迟剧烈波动

Part 02 重塑：基于 Kthena 的 LLM 声明式编排



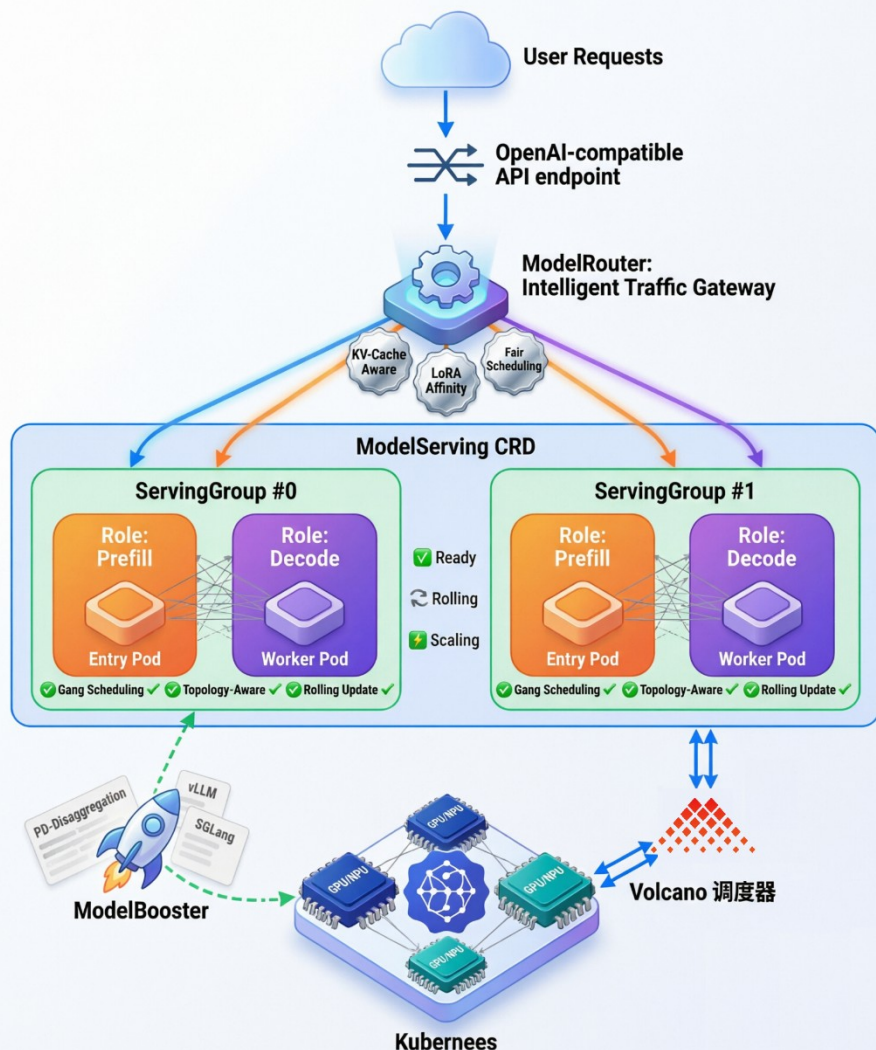
破局之道：Kthena + Volcano 联合底座



- Kthena 作为高性能大模型推理编排与流量调度大脑，提供推理编排，流量调度与扩缩容控制
- Volcano 提供强大的异构算力管理与高级调度算法，通过拓扑自动发现与亲和性调度提供最优调度方案
- 将分布式推理单元建模为具备拓扑约束的原子调度与扩缩容单元



Kthena ModelServing : 专为 LLM 设计的负载建模



• ModelServing (模型服务层) :

- 定义整体推理服务，管理多个 ServingGroup 实例
- 支持全局拓扑感知调度与 Gang Scheduling (协同调度)

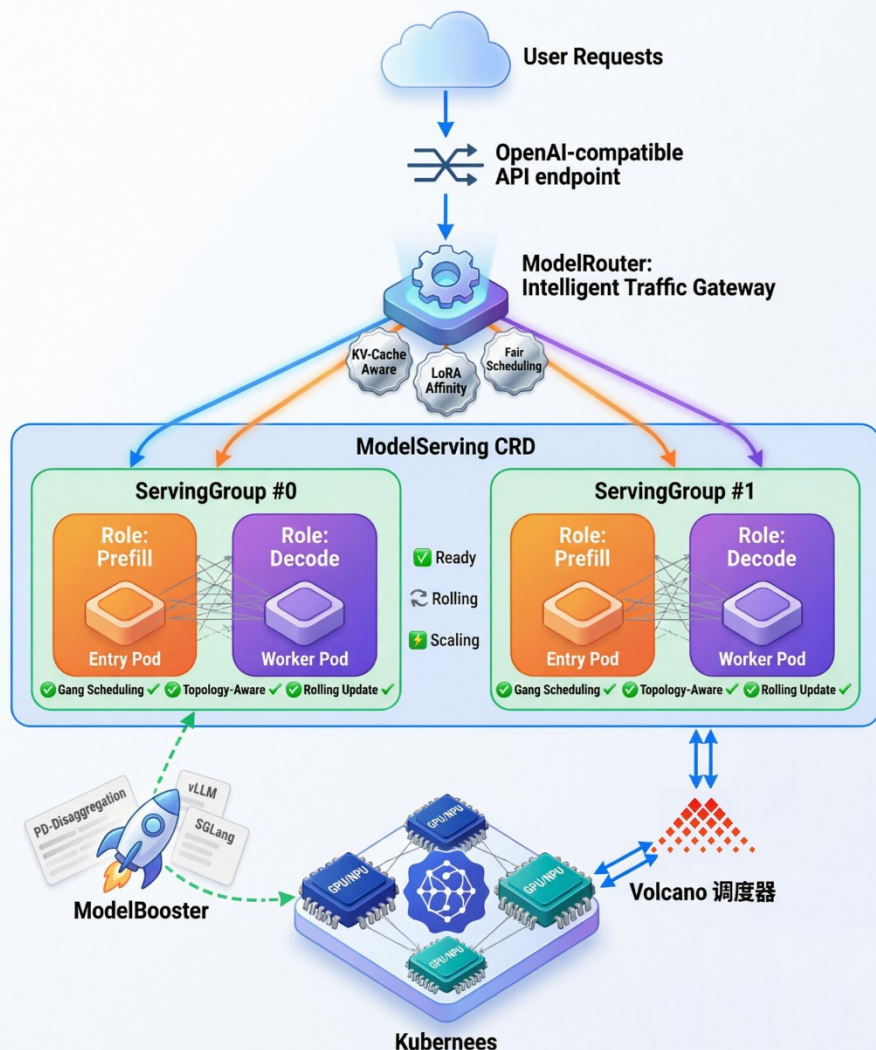
• ServingGroup (服务组层) :

- 独立完成一次推理任务的最小单位 (如包含 Prefill 和 Decode 角色)
- 支持平滑重构，降低服务中断

• Role (角色层) :

- 定义具体的功能角色 (如计算节点、存储节点)
- 支持 Entry/Worker 双 Pod 模板，优化节点内通信效率

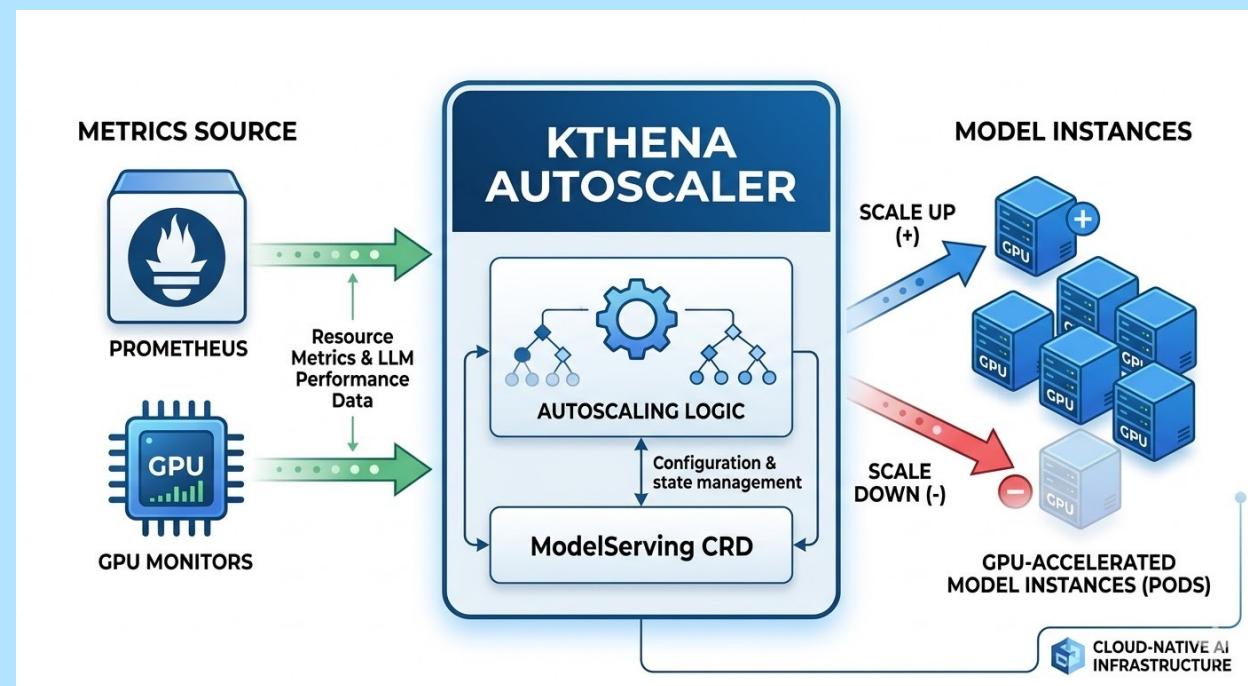
Kthena Router : 云原生 LLM 推理的智能流量枢纽



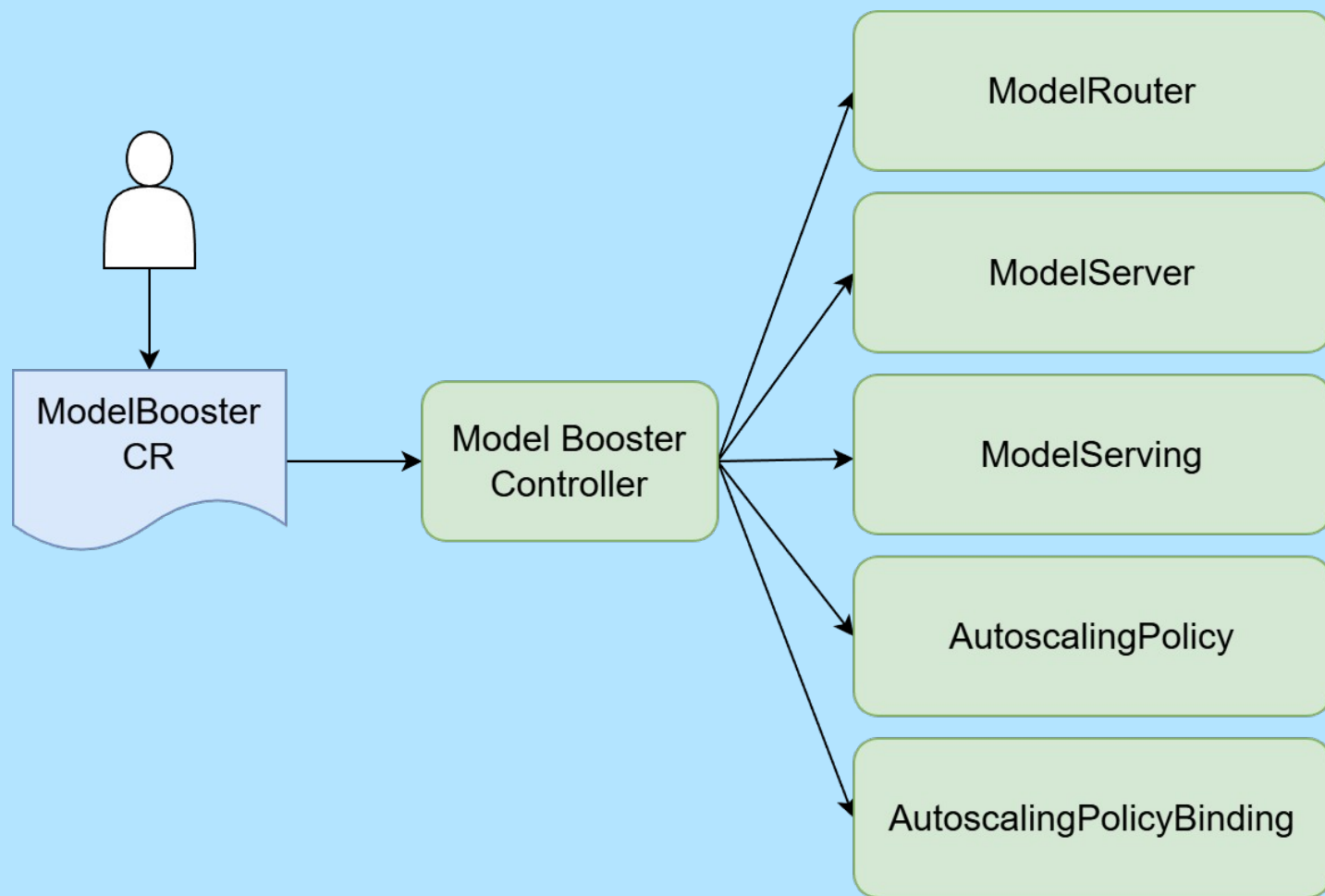
- 智能请求路由：支持基于模型名称、自定义 Header 或 URI 模式的精准转发。内置多种插件化评分算法（如 Least Request, Random 等）。
- **PD 分离**：原生支持将预填充（Prefill）和解码（Decode）阶段路由到不同的推理实例，极大提升硬件利用率并降低首字延迟（TTFT）。
- **KV Cache 感知**：通过 ScorePlugin 机制，优先将具有相同前缀的请求路由到存有对应 KV Cache 的节点，实验证明可提升吞吐量达 2.7 倍。
- 企业级流量管理：
 - 支持 LoRA 适配器 热插拔与按需路由。
 - 具备 Token 级限流、灰度发布及故障自动转移能力

Kthena Autoscaler：面向大模型推理的智能弹性伸缩架构

特性维度	传统 Kubernetes HPA	Kthena Autoscaler
指标感知	主要基于 CPU/ 内存利用率	深度感知 GPU 利用率、 KV Cache 占用、请求队列长度
推理角色拆分	无法识别内部组件，统一缩放	支持 PD 分离（ Prefill/Decode 角色独立伸缩）
异构资源支持	通常仅支持单一规格实例扩容	支持异构目标模式（在不同型号 GPU 间按成本 / 性能缩放）
缩放灵敏度	响应较通用，可能存在滞后	内置“恐慌模式”（ Panic Mode ），极速响应突发流量
调度协同	仅负责修改副本数	与 Volcano 深度集成，实现拓扑感知和最优任务放置
业务场景	通用微服务、Web 应用	专门针对大模型（ LLM ）推理、分布式架构优化



Kthena ModelBooster：极简一站式部署能力



- 关键特性：

- 极简部署：用户只需关心模型信息
- 自动协同：自动配置并同步子资源
- 状态追踪：全生命周期管理

- 现状与局限：

- 仅适配主流模型场景
- 不支持精细化速率限制与部分高级策略



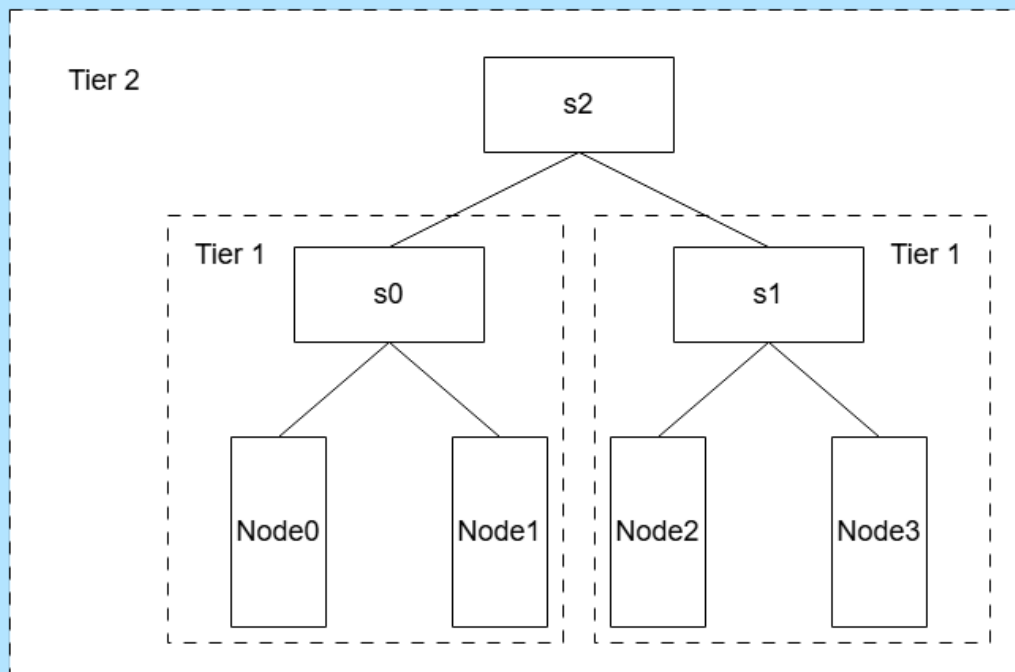
Part 03

深潜：多级拓扑感知与原子化 Gang 调度



Volcano 赋能：多级拓扑感知

- 网络感知：感知 TOR 交换机层级拓扑，将强通信依赖实例就近调度。
- 收益：最大化降低底层数据传输时延，保障吞吐量。



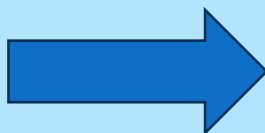
```
spec:
  replicas: 1 # servingGroup replicas
  template:
    networkTopology:
      rolePolicy:
        mode: hard
        highestTierAllowed: 1
      groupPolicy:
        mode: hard
        highestTierAllowed: 2
```

Volcano 赋能：原子化 Gang 调度

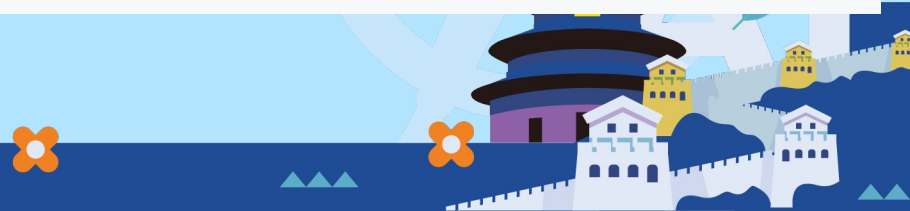
- 成组原子化：确保复杂角色实例同时启动，杜绝资源死锁占坑。

```
apiVersion: workload.serving.volcano.sh/v1alpha1
kind: ModelServing
metadata:
  name: sample
  namespace: default
spec:
  schedulerName: volcano
  replicas: 1 # servingGroup replicas
  template:
    restartGracePeriodSeconds: 60
    gangPolicy:
      minRoleReplicas:
        prefill: 2
        decode: 2
    roles:
      - name: prefill
        replicas: 4
        # ... additional role configuration
      - name: decode
        replicas: 4
        # ... additional role configuration
```

自动生成



```
apiVersion: scheduling.volcano.sh/v1beta1
kind: PodGroup
metadata:
  name: {modelserving-name}-{servinggroup-index}
  namespace: {modelserving-namespace}
  labels:
    modelinfer.volcano.sh/name: {modelserving-name}-{servinggroup-index}
  annotations:
    scheduling.k8s.io/group-name: {modelserving-name}
spec:
  # MinTaskMember defines minimum pods required for each role replica
  minTaskMember:
    "prefill-0": 2
    "decode-0": 2
    # ... additional role replicas
  minResources:
    # Aggregated resource requirements
  ttlSecondsAfterFinished: {timeout-seconds}
```

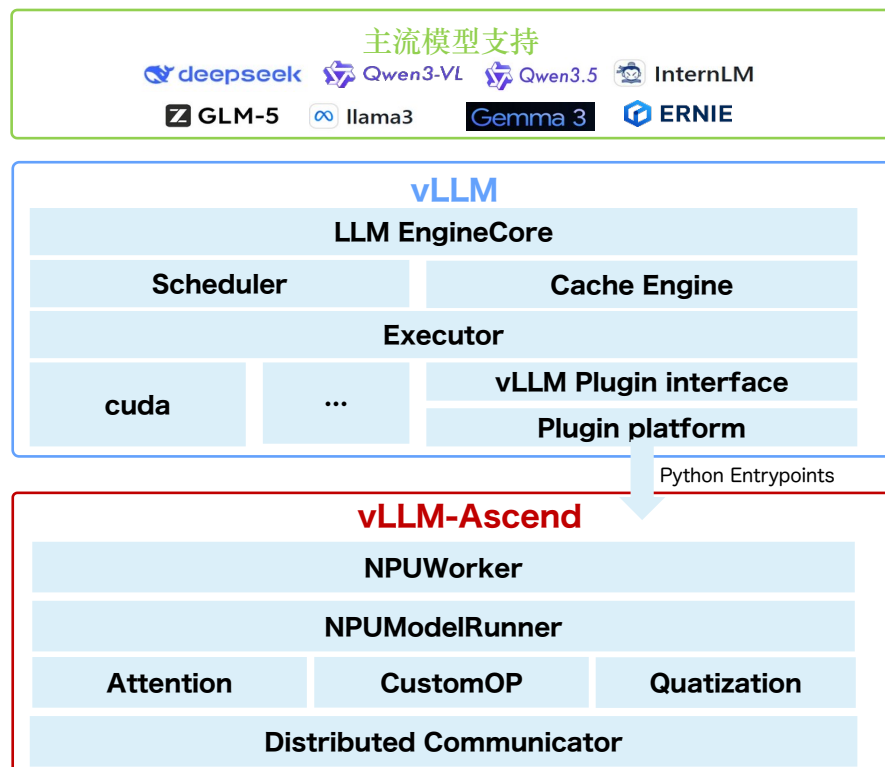


Part 04

实践：基于 vLLM-Ascend 的生产实践



vLLM Ascend：让 vLLM 在 Ascend 无缝运行的后端插件LM



项目核心数据



- 安装预构建 **wheel**
pip install vllm==0.13.0 vllm-ascend==0.13.0
- 容器一键拉起
quay.io/ascend/vllm-ascend:latest

<https://github.com/vllm-project/vllm-ascend>



vLLM-Ascend + Kthena 生产级架构

调度层

Kubernetes + Volcano

 拓扑感知调度
感知 TOR 交换机层级，实例就近放置

 Gang Scheduling
成组原子化调度，消除资源死锁

 资源管理
Ascend NPU 资源调度

编排层

Kthena

 ModelServing
定义 Prefill/Decode 角色

 ModelServer
管理 PD 分组与内部路由

 ModelRoute
暴露稳定的模型端点

 Autoscaler
KV Cache 感知扩缩容

推理引擎

vLLM × Ascend

 PD 分离
xPyD 架构，Mooncake KV 传输

 并行策略
TP/PP/EP/DP/Context Parallel

 推测解码
DeepSeek MTP 支持，量化

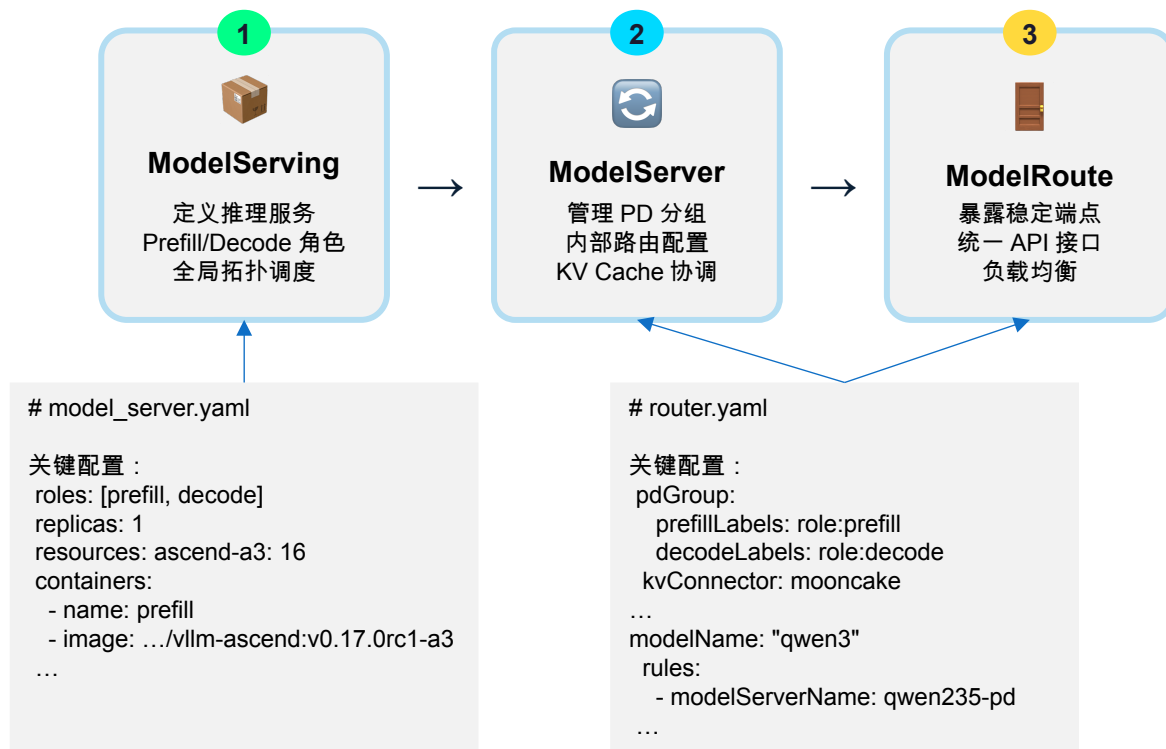
硬件层

Huawei Ascend NPU

 Atlas 800I A2/A3
高性能推理服务器

 RDMA 网络
低延迟 KV Cache 传输

vLLM Ascend 部署实践



Qwen3-235B 双机推理部署 (16 卡 * 2 机)

Step 1: 创建启动脚本 ConfigMap

```
kubectl apply -f config.yaml -n vllm-project
```

Step 2: 创建 ModelServing 工作负载 (包含 Headless Service)

```
kubectl apply -f model_server.yaml -n vllm-project
```

Step 3: 创建 ModelServer 和 ModelRoute (路由层)

```
kubectl apply -f router.yaml -n vllm-project
```

**128
5**

吞吐量 (tok/s)
峰值 1600
tok/s

**3.4
s**

首 Token 延迟
(TTFT)
P99: 5.0s

**63.7m
s**

每 token 延迟
(TPOT)
P99: 64.1ms

**100
%**

请求成功率
Gang 调度保
障



Thanks.

